
AZURE DATABRICKS

COMPLETE NOTES

≈ (The Lakehouse Platform) ≈



BEGINNER TO ADVANCED

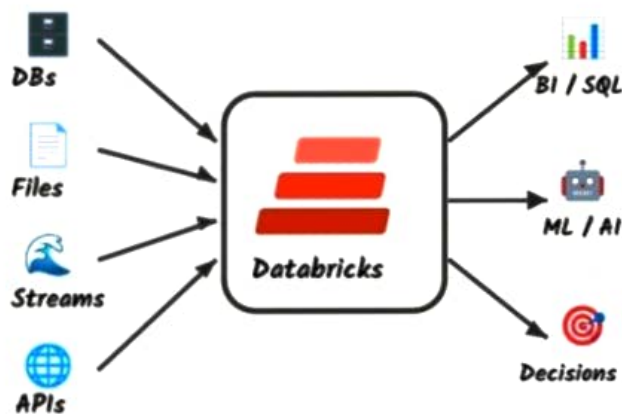
SWIPE TO NEXT →

① What is Big Data Engineering?

• The practice of **collecting, storing, processing & serving** huge volumes of data reliably so analysts & ML models can turn it into **business value**.

② What is Azure Databricks?

Azure Databricks is a **unified Lakehouse platform** — built on **Apache Spark** by the original Spark creators, offered as a first-party **Microsoft Azure** service. One platform to **ingest, transform, analyze & do ML** on all your data.



All data → One Lakehouse → Every use case!

💡 Lakehouse = Lake + Warehouse

Cheap open storage of a **data lake** + ACID reliability & SQL speed of a **warehouse** — thanks to **Delta Lake**.

③ Why Organizations Use Azure Databricks?

- ✓ One platform for **ETL + SQL + ML + streaming**
- ✓ Massive scale — Spark clusters on demand
- ✓ Open formats (Delta, Parquet) — no lock-in
- ✓ Native Azure integration (AD, ADLS, ADF, Power BI)
- ✓ Reliable pipelines with ACID Delta Lake
- ✓ Collaborative notebooks for teams
- ✓ Governance built-in (Unity Catalog)
- ✓ Pay-per-use clusters that auto-terminate

④ Key Features

- ⚡ **Apache Spark** — distributed compute engine at the core.
- ▲ **Delta Lake** — ACID tables on your data lake.
- 📓 **Notebooks** — Python, SQL, Scala & R together.
- 📁 **Unity Catalog** — central governance & lineage.
- 🔄 **Workflows & DLT** — orchestrate & automate pipelines.
- 🤖 **MLflow** — track, register & deploy ML models.

⑤ Real-world Applications

- 🏦 **Banking** — fraud detection on streaming transactions.
- 🛒 **Retail** — demand forecasting & recommendation engines.
- 🏥 **Healthcare** — patient data lakehouse & analytics.
- 🏭 **Manufacturing** — IoT sensor pipelines & predictive maintenance.
- 📺 **Media** — clickstream analytics & personalization.
- ✈️ **Airlines** — pricing, ops & loyalty analytics at scale.

★ Interview Tip:

Be ready for "Why Databricks over plain Azure Synapse / HDInsight?" — unified lakehouse, best Spark performance (Photon), Delta Lake, collaborative notebooks & open formats.

The Lakehouse Platform

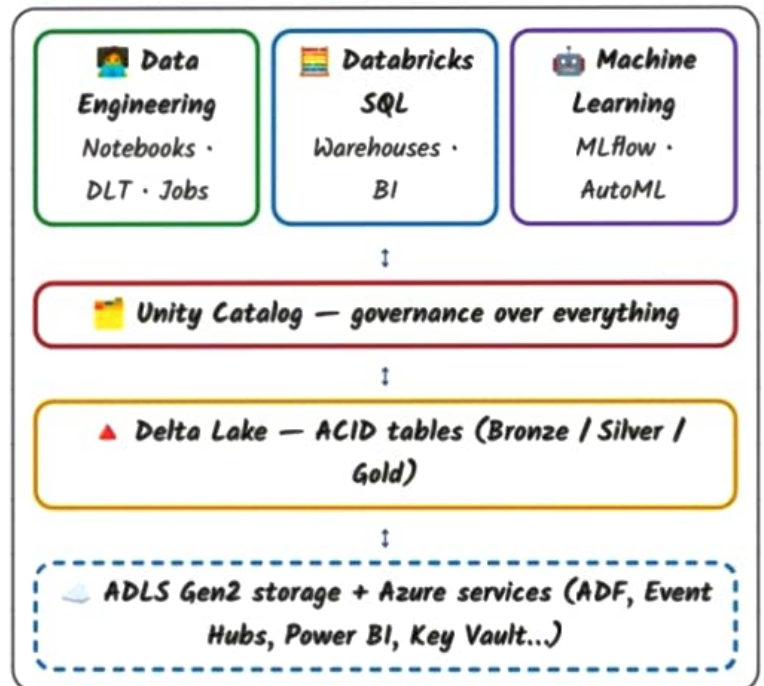
What is it?

Azure Databricks bundles everything a data team needs into one workspace – compute, storage access, pipelines, SQL, governance & ML – all on top of open Delta Lake storage in ADLS Gen2.

Key Components

-  **Workspace** – the web UI: notebooks, data, jobs, ML – organized per team.
-  **Clusters** – managed Spark compute (all-purpose & job clusters).
-  **Notebooks** – multi-language, collaborative, versioned via Repos/Git.
-  **Delta Lake** – ACID table format over Parquet files.
-  **Unity Catalog** – catalogs, permissions, lineage, sharing.
-  **Databricks SQL** – warehouses, queries, dashboards, alerts.
-  **Workflows / DLT** – job orchestration & declarative pipelines.
-  **MLflow + Feature Store** – the ML lifecycle end to end.

How They Fit Together



One copy of data – three personas working on it!

Pricing & Tiers (how you pay)

 DBU (Databricks Unit)	Standard vs Premium tier	Cluster cost formula
Unit of processing capability per hour – every workload consumes DBUs.	Premium adds RBAC, Unity Catalog, audit logs, SQL features. Most enterprises → Premium.	Azure VM cost + DBUs × rate. Job clusters & spot VMs are much cheaper!

💡 Key Takeaway:

Engineer in notebooks/DLT, **analyze** in DBSQL, **model** with MLflow – all governed by Unity Catalog on one Delta lakehouse.

★ Interview Tip:

Know that Azure Databricks is a **first-party Azure service** – billed via Azure, secured with **Entra ID (Azure AD)**, deployed into your subscription's VNet.

Architecture & End-to-End workflow

From raw data to decisions – the Lakehouse way!

Steps Explained

- ① **Data Sources**
Databases, files, IoT streams, SaaS apps, APIs.
- ② **Ingest**
ADF pipelines, Event Hubs / Kafka, Auto Loader for landing files.
- ③ **Store (ADLS Gen2)**
Cheap, durable cloud object storage – the "lake".
- ④ **Transform (Spark)**
Clean & enrich through Bronze → Silver → Gold Delta tables (Medallion).
- ⑤ **Govern**
Unity Catalog: permissions, lineage, audit.
- ⑥ **Serve – SQL & BI**
SQL Warehouses power dashboards & Power BI.
- ⑦ **Serve – ML & AI**
Train, register & deploy models with MLflow.
- ⑧ **Orchestrate**
Workflows / DLT schedule & monitor it all.
- ⑨ **Business Users**
Consume dashboards, apps & predictions.

① Sources

② Ingest — ADF / Auto Loader

③ ADLS Gen2 (the lake)

④ Bronze — raw Delta

⑤ Silver — clean & conformed

⑥ Gold — business aggregates

⑦ SQL Warehouse / MLflow

⑧ Dashboards • Apps • Models

⑨ Decisions

Control vs Data Plane

Control plane (Databricks-managed): web UI, notebooks, jobs config, cluster manager.
Data plane (YOUR Azure subscription): cluster VMs + storage — your data never leaves your tenant.

Key Points

- ADLS Gen2 = cheap open storage
- Delta = reliability layer
- Medallion = quality layers
- Spark = distributed muscle
- UC = one governance layer
- Workflows = automation
- DBSQL / MLflow = serving

★ Interview Tip

Whiteboard this flow: **Source** → **Ingest** → **Lake** → **Bronze/Silver/Gold** → **Serve (BI + ML)**. Then mention control-vs-data plane — it instantly signals real Azure Databricks understanding!

Getting Data Into the Lakehouse

Common Sources & Ingestion Paths

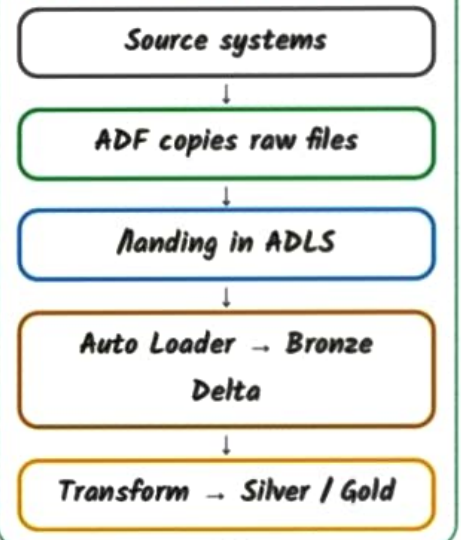
-  **ADLS Gen2 / Blob** — main landing zone for files.
-  **Azure SQL / Synapse / Cosmos** — via JDBC or ADF copy.
-  **Event Hubs / Kafka** — real-time streams.
-  **Azure Data Factory** — 100+ connectors, copy into the lake, then trigger Databricks.
-  **Auto Loader** — incremental file ingestion (cloudFiles).
-  **COPY INTO** — SQL command for idempotent batch loads.
-  **Delta Sharing / Marketplace** — data from partners.
-  **Sample datasets** — /databricks-datasets for practice!

Accessing storage — the modern way

```
# Unity Catalog external location (best practice)
df = spark.read.parquet("abfss://raw@mylake.../sales/")

# Legacy: DBFS mounts (dbutils.fs.mount) - being phased out;
# prefer UC Volumes & external locations with Managed Identity.
```

Typical Landing Flow



Auto Loader vs COPY INTO vs ADF

Aspect	Auto Loader 	COPY INTO 	ADF Copy 
Style	Streaming (incremental, cloudFiles)	SQL batch command	GUI pipelines outside Databricks
Best at	Millions of small files, continuous arrival	Simple scheduled folder loads	Pulling from 100+ external systems
Schema	Inference + evolution + rescue column	Basic inference	Mapping in pipeline
State	Checkpoint tracks processed files	Idempotent by file	Pipeline runs
Use when	Files land continuously in the lake	Occasional batch folders	Source is outside Azure storage

Tips

- ✓ Land raw files first, then ingest — keeps a replayable history.
- ✓ Use **Managed Identity / service principals**, never account keys in code.
- ✓ Store secrets in **Key Vault-backed secret scopes**.

Interview Tip

Classic combo question: "ADF vs Databricks?" — ADF = ingestion & orchestration glue; Databricks = heavy transformation, streaming & ML. They complement, not compete.

Workspace & Clusters (Compute)

The Workspace

- **Sidebar** — Workspace (notebooks), Catalog (data), Workflows (jobs), Compute (clusters), SQL, ML.
- **Repos** — Git-backed folders (GitHub / Azure DevOps) for CI/CD.
- **Catalog Explorer** — browse catalogs → schemas → tables, preview data & lineage.
- **Admin console** — users, groups, cluster policies, workspace settings.

Cluster anatomy

A cluster = 1 driver + N workers (Azure VMs) running a **Databricks Runtime (DBR)** — Spark + optimizations + libraries. Choose DBR LTS versions for stability; **Photon** runtime accelerates SQL/DataFrame workloads.

```
# Cluster essentials to configure
Access mode : Single user / Shared (UC)
Workers      : 2-8 (autoscaling range)
VM type      : Memory / Compute / Storage
Auto-term    : 30 min ← always set!
Spot: workers only → big savings
```

All-purpose vs Job Clusters

All-purpose 	Job cluster 
For interactive notebooks & exploration	Spun up by a job, killed when done
Shared by team, long-lived	Fresh & isolated per run
More expensive per DBU	Much cheaper per DBU
Dev & debugging	Production pipelines

Other compute options

- **SQL Warehouses** — for Databricks SQL (serverless = instant start).
- **Instance Pools** — pre-warmed VMs to cut cluster start time.
- **Single-node** — small data / ML experimentation.
- **Serverless compute** — Databricks manages the VMs entirely.

Cost habits

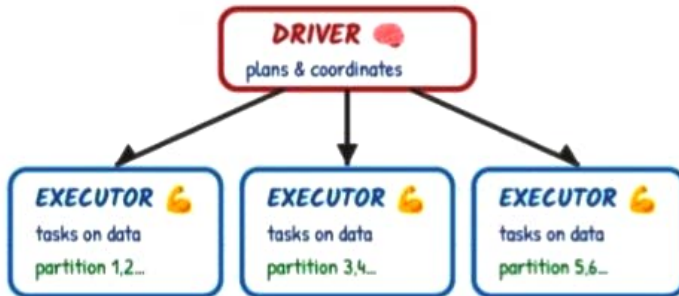
- ✓ Auto-terminate every cluster (30 min or less).
- ✓ Prod jobs → job clusters + spot workers.
- ✓ Use cluster policies to enforce limits for teams.

★ Interview Tip

"How do you control Databricks costs?" — job clusters, auto-terminate, autoscaling, spot instances, pools for start-time, Photon for speed, cluster policies for guardrails. Name five and you've nailed it.

Apache Spark Fundamentals

How Spark Executes 🔥



Data is split into **partitions**; the driver turns your code into **jobs** → **stages** → **tasks** and distributes them to executors in parallel.

Lazy evaluation ⚡

Transformations only **build a plan** (DAG). Nothing runs until an **action** triggers it – Letting Catalyst optimize the whole plan first.

Transformations vs Actions

Transformations 🔄 (lazy)

select, filter, withColumn,
join, groupBy, orderBy,
union, distinct

Actions ▶️ (execute)

show, count,
collect, first, write,
display, toPandas

Narrow vs Wide (the shuffle!)

Narrow 🔍

Row stays in its
partition: filter,
select, withColumn

Cheap

Wide 🔄 (shuffle)

Rows move between
partitions: groupBy, join,
orderBy, distinct

Expensive – network +
disk. Minimize!

```
# PySpark in a notebook - spark session
df = spark.read.table("nyc.trips")
out = (df.filter("fare_amount > 0")
        .groupBy("pickup_zip")
        .count()) # still lazy...
out.display()    # action → runs!
```

💡 Mental model

RDD (low-level) → **DataFrame** (named columns, optimized – use this!) → SQL (same engine). All compile to the same Catalyst plans.

★ Interview Tip

"Why is Spark fast?" – in-memory processing, lazy DAG optimization (Catalyst), whole-stage codegen (Tungsten/Photon) & massive parallelism.

Notebooks, Magic Commands & dbutils

Magic Commands (%) — switch language per cell

`%python / %sql / %scala / %r` — run the cell in that language; mix freely in one notebook.

`%md` — markdown documentation cells (headings, images, links).

`%run /path/nb` — execute another notebook inline (share functions & variables).

`%fs ls /databricks-datasets` — quick filesystem commands.

`%sh` — shell commands on the driver node.

`%pip install pkg` — notebook-scoped Python libraries.

dbutils — the Swiss-army knife

Module	What it does	Example
<code>dbutils.fs</code>	Filesystem ops on DBFS / volumes	<code>dbutils.fs.ls("/Volumes/main/raw/")</code>
<code>dbutils.widgets</code>	Notebook parameters (text, dropdown)	<code>dbutils.widgets.text("env", "dev")</code>
<code>dbutils.secrets</code>	Read Key-Vault-backed secrets safely	<code>dbutils.secrets.get("kv-scope", "sql-pwd")</code>
<code>dbutils.notebook</code>	Chain notebooks with params & return values	<code>dbutils.notebook.run("./child", 300, {"env": "prod"})</code>
<code>dbutils.jobs</code>	Task values between job tasks	<code>taskValues.set("rows", 500)</code>

Collaboration superpowers

- ✓ Real-time co-editing & comments (like Google Docs).
- ✓ **Repos** — sync notebooks with GitHub / Azure DevOps for PRs & CI/CD.
- ✓ Version history & revert built in.
- ✓ `display(df)` — instant tables & charts; **Databricks Assistant** for AI help.

★ Interview Tip

`"%run vs dbutils.notebook.run?"` — `%run` inlines the notebook (shares state/variables); `notebook.run` starts a **separate job** with its own scope, takes parameters & returns a value.

💡 Tip

Parameterize every prod notebook with **widgets** (`env`, `date`, `path`) — same code runs in dev & prod.

DataFrames — Read → Transform → Write

Read 📖

```
df = (spark.read
      .format("csv")           # csv|json|delta
      .option("header", True)
      .option("inferSchema", True)
      .load("abfss://raw@lake.../sales/"))

# or from a governed table
df = spark.read.table("main.sales.orders")
```

Write 💾

```
(df.write
  .format("delta")
  .mode("overwrite") # append|overwrite
  .saveAsTable("main.sales.orders_clean"))
```

Always land as Delta — you get ACID, time travel & fast reads for free.

Transform ↻ (the daily toolkit)

```
from pyspark.sql import functions as F

out = (df
  .select("id", "amount", "city", "order_ts")
  .filter(F.col("amount") > 0)
  .withColumn("order_date",
              F.to_date("order_ts"))
  .withColumn("tier",
              F.when(F.col("amount") > 1000, "High")
                .otherwise("Normal"))
  .dropDuplicates(["id"])
  .groupBy("city", "tier")
  .agg(F.sum("amount").alias("total"),
       F.countDistinct("id").alias("orders")))
```

Joins & more

```
a.join(b, "customer_id", "left")
df.orderBy(F.desc("total"))
df.createOrReplaceTempView("v") # then %sql on i
```

Handy function families

Family	Favorites
Strings 	concat, split, substring, regexp_replace, lower/upper, trim, lpad
Dates 	to_date, date_add, datediff, date_trunc, year/month/dayofweek, current_date
Nulls 	coalesce, isNull, na.fill, na.drop, nullif
Windows 	row_number/rank over Window.partitionBy("cust").orderBy(F.desc("ts")) — dedup & top-N!
Semi-structured 	explode, from_json, get_json_object, struct, array functions

★ Interview Tip

Know the dedup pattern cold: `row_number()` over a window partitioned by key, ordered by latest timestamp, keep `rn=1` — it comes up in nearly every data engineering interview!

Delta Lake — the Heart of the Lakehouse

What is it?

Delta Lake = Parquet data files + a transaction log (`_delta_log`) that adds database-grade reliability to cheap lake storage. Open source, default format in Databricks.

Superpowers

- ✓ **ACID transactions** — no more half-written data.
- ✓ **Time travel** — query any historical version.
- ✓ **Schema enforcement** — rejects bad-shape writes.
- ✓ **Schema evolution** — mergeSchema when you mean it.
- ✓ **MERGE** — upserts & CDC made easy.
- ✓ **Streaming + batch** — same table serves both.
- ✓ **DML** — UPDATE / DELETE / MERGE on a lake!

Maintenance commands

```
OPTIMIZE sales ZORDER BY (customer_id);  
-- compacts small files + co-locates data  
VACUUM sales RETAIN 168 HOURS;  
-- deletes old unreferenced files  
DESCRIBE HISTORY sales; -- audit trail
```

Time travel

```
SELECT * FROM sales VERSION AS OF 12;  
SELECT * FROM sales  
  TIMESTAMP AS OF "2026-08-01";  
RESTORE TABLE sales TO VERSION AS OF 12;
```

MERGE (upsert)

```
MERGE INTO customers t  
USING updates s  
  ON t.id = s.id  
 WHEN MATCHED THEN UPDATE SET *  
 WHEN NOT MATCHED THEN INSERT *;
```

Under the hood

Every write adds a JSON commit to `_delta_log`; readers replay the log to see a consistent snapshot. Checkpoints every 10 commits keep it fast.

Delta vs Plain Parquet

Feature	Parquet	Delta
ACID / concurrent writes	✗	✓
UPDATE / DELETE / MERGE	✗ (rewrite files yourself)	✓ SQL DML
Time travel / history	✗	✓ versions + RESTORE
Schema management	None	Enforcement + evolution
Streaming source/sink	Limited	First-class

★ Interview Tip

Star answer: "Delta = Parquet + transaction log." Then name ACID, time travel, MERGE & OPTIMIZE/Z-ORDER. Bonus: VACUUM default retention is **7 days** — protects time travel from deleting too soon.

Medallion Architecture



Progressively refine data quality, layer by layer!



Layer	What lives here	Typical work	Who uses it
Bronze	Raw data exactly as received + metadata (ingest time, source file). Append-only history.	Auto Loader ingestion, schema capture, rescue bad records.	Engineers (replay & audit)
Silver	Validated, deduped, typed, conformed entities (customers, orders...). Joinable "single source of truth".	Cleansing, dedup windows, MERGE upserts/CDC, quality checks (DLT expectations).	Engineers, scientists, analysts
Gold	Business-level star schemas, KPIs & feature tables – optimized for consumption.	Aggregations, dimensional modeling, Z-ORDER for BI speed.	BI dashboards, execs, ML serving

Why layers? 🤔

- ✓ **Replayability** – rebuild Silver/Gold anytime from Bronze.
- ✓ **Debuggability** – trace a bad KPI back layer by layer.
- ✓ **Separation of concerns** – ingestion vs quality vs business logic.
- ✓ **Access control** – analysts see Gold; raw PII stays locked in Bronze/Silver.
- ✓ **Incremental everything** – each hop can be streaming.

Naming in Unity Catalog

```
main.bronze.orders_raw
main.silver.orders
main.gold.daily_sales_kpi
```

💡 Tip

Don't over-engineer: small projects can start Bronze → Gold and add Silver when complexity grows.

★ Interview Tip

Medallion is the **most-asked Databricks design question**. Practice one concrete story: "IoT JSON lands via Auto Loader in Bronze → dedup + type in Silver with MERGE → hourly device KPIs in Gold for Power BI."

Structured Streaming & Auto Loader

The Big Idea 🌊

Structured Streaming treats a stream as an **infinite table** — you write the same DataFrame code, Spark runs it incrementally in micro-batches, and a **checkpoint** remembers exactly what's been processed (exactly-once!).

Read a stream 🏠

```
# Auto Loader: incremental files from the lake
df = (spark.readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.schemaLocation",
    "/Volumes/main/chk/schema")
  .load("abfss://landing@lake.../events/"))
```

Write a stream 📁

```
(df.writeStream
  .option("checkpointLocation",
    "/Volumes/main/chk/bronze_events")
  .outputMode("append")
  .trigger(availableNow=True) # batch-style!
  .toTable("main.bronze.events"))
```

Why Auto Loader rocks 🏠

- ✓ Discovers **only new files** — scales to millions.
- ✓ **Schema inference + evolution**; bad data lands in **_rescued_data** instead of failing.
- ✓ Exactly-once via checkpoints.
- ✓ Same code for continuous or scheduled (**availableNow**) runs.

Key knobs 🛠️

- **Triggers** — availableNow (drain & stop) • processingTime="1 minute" • continuous.
- **Output modes** — append (most common) • complete (aggregations) • update.
- **Watermarks** — bound state for late data: withWatermark("ts", "10 minutes").
- **Checkpoint per sink** — never share or delete casually (that resets the stream!).

Batch vs Streaming — choosing

Batch 🏠 (scheduled)	Streaming 🌊 (incremental)
Data arrives on a schedule; latency of hours is fine; simplest ops.	Files/events arrive continuously; minutes-or-less latency; exactly-once bookkeeping handled for you.
Reports, daily loads	CDC, IoT, clickstream, fraud, Bronze ingestion

★ Interview Tip

Golden line: "I use Auto Loader with trigger **availableNow**" — streaming reliability (checkpoints, schema evolution) at batch cost, the recommended modern ingestion pattern.

Unity Catalog — Governance Layer

Three-level namespace 📁

Metastore 🏠 → one per region, attached to workspaces

└ Catalog 📁 (main, dev, prod...)

└ Schema 📁 (bronze, sales...)

└ Tables • Views • Volumes • Functions • Models

Query as: `catalog.schema.table`

Managed vs External tables

Managed 🏠	External 🔗
UC owns data + metadata; DROP deletes files. Default & recommended.	Points at your ADLS path; DROP keeps files. For shared/legacy data.

What UC gives you 📁

- ✓ **Central permissions** — GRANT/REVOKE once, applies across all workspaces.
- ✓ **Lineage** — auto column-level, table → notebook → job.
- ✓ **Audit logs** — who accessed what, when.
- ✓ **Data discovery** — search, tags, certifications.
- ✓ **Volumes** — governed non-tabular files.
- ✓ **Delta Sharing** — share live data cross-org, no copies.
- ✓ **Row filters & column masks** — fine-grained security.

```
GRANT SELECT ON main.gold.daily_kpi  
TO `analysts`;  
GRANT USE CATALOG ON CATALOG main  
TO `analysts`;
```

Unity Catalog vs legacy hive_metastore

Aspect	hive_metastore 🏠	Unity Catalog 📁
Scope	Per workspace	Across all workspaces in a region
Namespace	schema.table (2 levels)	catalog.schema.table (3 levels)
Security	Cluster-bound, patchy	Central ANSI GRANTS, row/column-level
Lineage / audit	None	Built-in

💡 Key Takeaway

UC = **one catalog, one permission model, full lineage** over every table, file, function & model — the backbone of enterprise Databricks.

★ Interview Tip

Expect "How do you secure data in Databricks?" — UC grants + row filters/column masks + workspace isolation via catalogs (dev/test/prod) + audit logs.

Databricks SQL & Serving BI

SQL Warehouses

Dedicated compute for SQL analytics — powered by **Photon** for warehouse-class speed on your Delta tables.

Type	When
Serverless 	Instant start, auto-manage — best default for BI.
Pro	Full features, compute in your account.
Classic	Basic, cheapest, slower start.

What you build in DBSQL

- ✓ **Queries** — SQL editor with autocomplete & history.
- ✓ **Dashboards** — charts & filters straight on Gold tables.
- ✓ **Alerts** — email/Teams when a metric crosses a threshold.
- ✓ **Genie** — natural-language Q&A over governed data.

★ Interview Tip

"Why not just use Synapse / Fabric for BI?" — with DBSQL your BI runs **directly on the same governed Delta tables** your pipelines produce: no copies, no sync jobs, one security model (and Photon makes it fast).

Power BI connection

- ① Power BI Desktop → Get Data → **Azure Databricks** connector.
- ② Paste the warehouse's **Server hostname + HTTP path**.
- ③ Sign in with **Entra ID** (respects UC permissions!).
- ④ Import or **DirectQuery** the Gold tables.

```
-- classic BI-serving pattern
CREATE VIEW main.gold.v_sales AS
SELECT region, order_date,
       SUM(amount) AS revenue,
       COUNT(DISTINCT order_id) AS orders
FROM main.gold.fact_sales
GROUP BY region, order_date;
```

Tip

Serve BI from **Gold views** — rename/reshape without breaking reports; OPTIMIZE + Z-ORDER the base tables for snappy dashboards.

Workflows, Jobs & Delta Live Tables

Workflows (Jobs) 🔄 — production orchestration

- A **job** = DAG of **tasks**: notebooks, Python scripts/wheels, SQL, dbt, DLT pipelines.
- **Dependencies** — task B after A; branch with if/else & for-each.
- **Triggers** — cron schedule, file arrival, continuous, or API/ADF call.
- **Reliability** — retries, timeouts, email/Teams alerts on failure.
- Run on cheap **job clusters**; pass values via **taskValues** & job parameters.



One job DAG, alerts on failure, all on job clusters!

Delta Live Tables (DLT) ✨ — declarative pipelines

```
import dlt

@dlt.table(comment="clean orders")
@dlt.expect_or_drop("valid_amount",
                    "amount > 0")
def silver_orders():
    return (dlt.read_stream("bronze_orders")
            .dropDuplicates(["order_id"]))
```

- ✓ You declare **what** tables should exist — DLT figures out order, clusters & retries.
- ✓ **Expectations** = built-in data quality rules (warn / drop / fail) with metrics.
- ✓ Handles **incremental + streaming** automatically; auto-generated lineage graph.
- ✓ Great fit for **Medallion** pipelines.

ADF vs Workflows vs DLT — who orchestrates what?

Tool	Sweet spot	Remember
ADF ⚙️	Cross-service Azure orchestration & 100+ source copies	Often triggers Databricks jobs/notebooks.
Workflows 🔄	Everything inside Databricks — cheapest & native	Modern default for lakehouse pipelines.
DLT ✨	Declarative Medallion pipelines with data quality	Runs as a task inside Workflows.

★ Interview Tip

"How do you productionize a notebook?" — parameterize with widgets → commit to Repos → run as a Workflows job on a job cluster with retries + alerts → promote dev/test/prod via CI/CD (Asset Bundles).

Machine Learning & MLflow

The ML Lifecycle on Databricks



MLflow building blocks

Piece	What it does
Tracking 	Logs params, metrics, artifacts per run — compare experiments side-by-side.
Models 	Standard packaging (flavor = sklearn, xgboost, pytorch...).
Registry 	Versioned models in Unity Catalog with aliases (@champion / @challenger).
Serving 	Real-time REST endpoints, autoscaling, monitored.

```
import mlflow
mlflow.autolog() # logs everything!

with mlflow.start_run():
    model.fit(X_train, y_train)
    mlflow.log_metric("auc", auc)
```

The toolbox

- ✓ **ML Runtime** — clusters preloaded with sklearn, XGBoost, PyTorch, TensorFlow.
- ✓ **AutoML** — click-to-baseline models with editable generated notebooks.
- ✓ **Feature Store** — governed, reusable features, consistent train/serve.
- ✓ **Vector Search + Model Serving** — RAG & GenAI apps on your lakehouse data.
- ✓ **Mosaic AI / Foundation Models** — hosted LLM endpoints.

Why lakehouse ML wins

Features come straight from governed **Silver/Gold Delta tables** — no export to a separate ML platform, full lineage from raw data to model.

★ Interview Tip

Even for data-engineer roles, know the handshake: **your Gold tables are the ML team's inputs** — quality expectations, timely refreshes & Feature Store keep models healthy.

Performance Optimization ⚡

Fast pipelines = low cost – same levers!

DO ✓

- ✓ Use **Photon** + recent DBR LTS.
- ✓ **OPTIMIZE** tables; enable **predictive optimization**.
- ✓ **Z-ORDER** / **Liquid Clustering** on common filter keys.
- ✓ **Broadcast** small dims in joins (auto <10MB, or hint).
- ✓ Filter early, select only needed columns.
- ✓ Let **AQE** handle skew & partition sizes (on by default).
- ✓ **cache()** a DataFrame reused many times (then unpersist).
- ✓ Partition big tables by low-cardinality date columns.

AVOID ✗

- ✗ **collect()** on big data – blows up the driver.
- ✗ Many **tiny files** – the classic lake killer (compact them!).
- ✗ Python **UDFs** when built-in functions exist (10-100x slower).
- ✗ Over-partitioning (thousands of small partitions).
- ✗ **MERGE** without pruning conditions on huge tables.
- ✗ Shuffles you don't need – **repartition()** sparingly.
- ✗ Oversized clusters "just in case" – scale from small.

Where is my job slow? 🔍

Spark UI →
longest stage

→ Check shuffle
read/write size

→ Look for skew
(1 slow task)

→ Fix: broadcast, salt
keys,
AQE, file compaction

Symptom	Likely cause	Fix
Slow reads	Small files / no pruning	OPTIMIZE, Z-ORDER, partition filters
One task lags	Data skew	AQE skew join, salting, broadcast
Driver OOM	collect() / toPandas() on big df	Aggregate first, limit, write to table
Huge shuffle	Wide ops on raw data	Filter/project early, pre-aggregate
Slow MERGE	Full-table scans	Partition/Z-ORDER on merge keys, tight ON clause

★ Interview Tip

Frame answers as a framework: **Storage (files, clustering)** → **Plan (filters, joins, AQE)** → **Compute (Photon, sizing)** → **Code (no collect, no needless UDFs)**. Structured beats scattered!